

Rapport du projet **Deforest**

Table des matières

Rapport du projet Deforest.....	1
I - Équipe du projet:.....	4
II - Espace de Travail et Outils:.....	4
III - Description du Sujet et des Données:.....	4
IV - Planification Globale et Méthodologie:.....	5
Organigramme des tâches:.....	5
Diagramme de PERT:.....	6
Diagramme de GANT:.....	8
- Outils mis en place sur le serveur.....	9
Architecture et Technologies déployées.....	9
Explication des points stratégiques du code.....	9
Analyse et Traitement des données.....	10
Défi technique : Les limites de l'API et le traitement différé.....	10
- Interface Utilisateur.....	11
V - Répartition des Tâches & Avancement.....	12
Description de l'état:.....	12
Tableau de Répartition:.....	12
VI - Rapports Personnels.....	13
Fernandez Mathys.....	13
Gestion de projet et versioning (GitLab):.....	13
Prise en main de l'API Global Forest Watch:.....	14
Traitement des données et affichage (Frontend):.....	14
Conclusion :.....	14
Ratsimbazafy Harinavalona.....	15
Soilihi Timéo.....	15
Mise en place du serveur et déploiement.....	15
Automatisation de l'environnement de travail.....	15
Amélioration de la carte et correction des bugs d'affichage.....	15
Ergonomie et interface utilisateur.....	16
Abbes Yiris.....	16
Zambrano Junior.....	16
Premier partie l'index.....	16
Deuxième partie global.css.....	17
Troisième partie carte.js et carte.css.....	17
Aballo Zaef.....	18
Présentation.....	18
Mise en place des marqueurs de déforestation.....	18
Mise en place du clustering des marqueurs.....	18
Sécurisation des clés API.....	19
Mise en place du backend Express.js.....	19
Documentation API.....	19
Utilisation de Git et GitLab.....	19
Conclusion.....	19
VII - Pourquoi ce cas d'utilisation ?.....	19
VIII – Referiez-vous le projet de la même manière ?.....	20

I - Équipe du projet:

Identité :

- Fernandez Mathys

- Soilihi Timéo

- Aballo Zaef

- Abbes Yiris

- Zambrano Junior

- Ratsimbazafy Harinavalona

Role :

Chef de projet | Gestionnaire API

Lead Developer | Gestionnaire serveur

Scrum Master | Gestionnaire API | Responsable Qualité/Tests

Gestionnaire serveur | Designer UI/UX

Designer UI/UX | Spécialiste Cartographie

Data Visualisation | Designer UI/UX

II - Espace de Travail et Outils:

Dépôt de code : [lienGitLab](#)

Gestion de projet : [lienGitLab](#)

Organisation de l'équipe : [lienGitLab](#)

III - Description du Sujet et des Données:

1. Sujet:

Le projet consiste à concevoir et développer une interface web interactive dédiée à la surveillance et à l'analyse de l'état des forêts. L'application se connecte à l'API publique de Global Forest Watch (GFW) pour récupérer les données environnementales en temps réel ou de manière historique. Ces informations seront restituées à l'utilisateur sous deux formes principales : une carte interactive pour l'analyse spatiale avec des recherches manuelles et un tableau de bord composé de graphiques pour l'analyse statistique.

2. Données utilisées :

1. Sources :

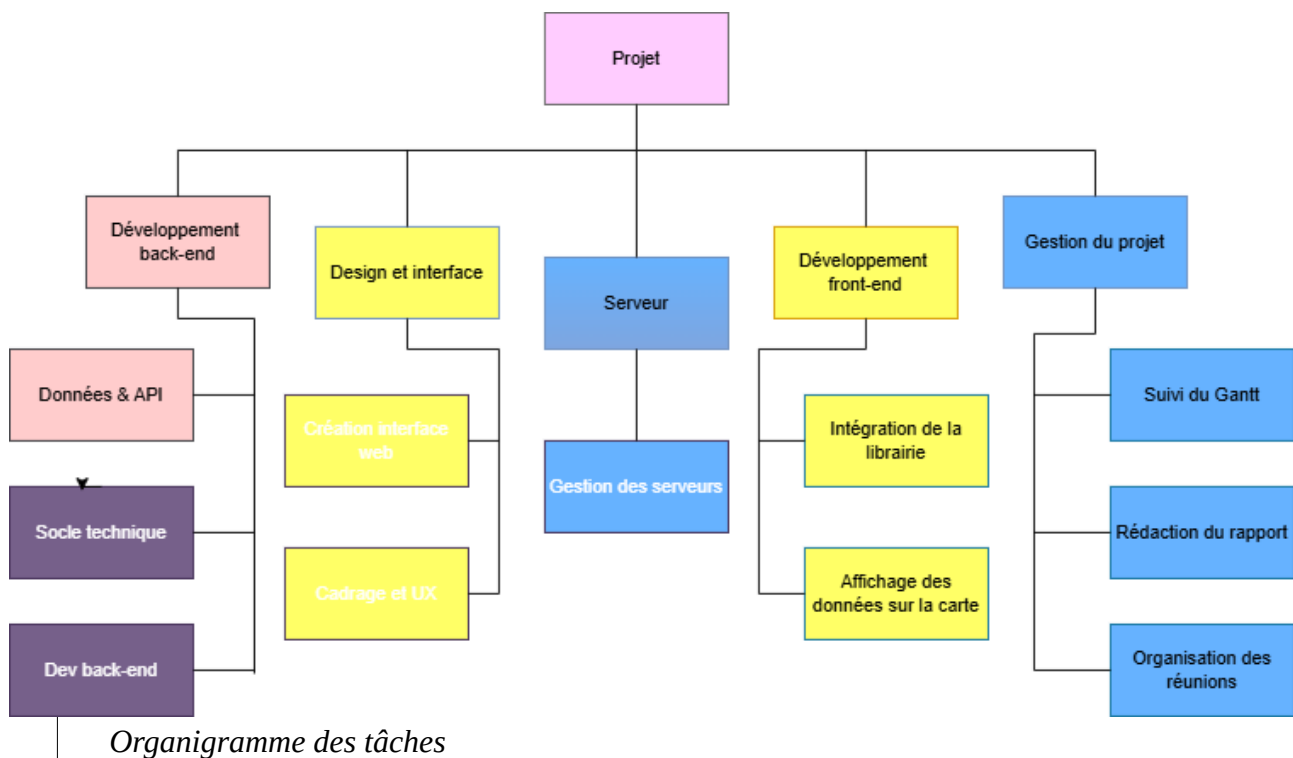
API publique de Global Forest Watch

(Utilisation de requête SQL avec la clé API enregistré dans le backend)

2. Format :

Les données seront principalement récupérées au format JSON directement à l'utilisateur. Ces données ne sont pas stockés sur le serveur mais temporairement dans la RAM de l'ordinateur de l'utilisateur.

IV - Planification Globale et Méthodologie:



Organigramme des tâches:

L'architecture s'articule autour du nœud central "Projet", qui se divise directement en cinq grands pôles d'expertise :

- **Développement back-end :**

Ce pôle technique regroupe trois sous-tâches essentielles : la gestion des **Données & API** (connexion à Global Forest Watch), la mise en place du **Socle technique** initial, et le **Dev back-end** (création du serveur Node.js et gestion des routes).

- **Design et interface :**

Dédié à la conception visuelle, ce bloc se concentre sur l'ergonomie avant le passage au code. Il inclut le **Cadrage et UX** (expérience utilisateur) et la **Création interface web** (structuration des maquettes).

- **Serveur :**

Un pilier d'infrastructure dédié spécifiquement à la **Gestion des serveurs**, afin d'assurer l'hébergement et la stabilité de notre environnement.

- **Développement front-end :**

Ce pôle opérationnel se charge de rendre l'interface interactive. Il est composé de l'**Intégration de la librairie** (Leaflet pour la carte, Chart.js pour les graphiques) et de l'**Affichage des données sur la carte** (placement dynamique des points de déforestation).

- **Gestion du projet :**

Ce pilier transversal est indispensable pour coordonner notre équipe. Il englobe le **Suivi du Gantt** (respect du planning), l'**Organisation des réunions**, et enfin la **Rédaction du rapport** de projet.

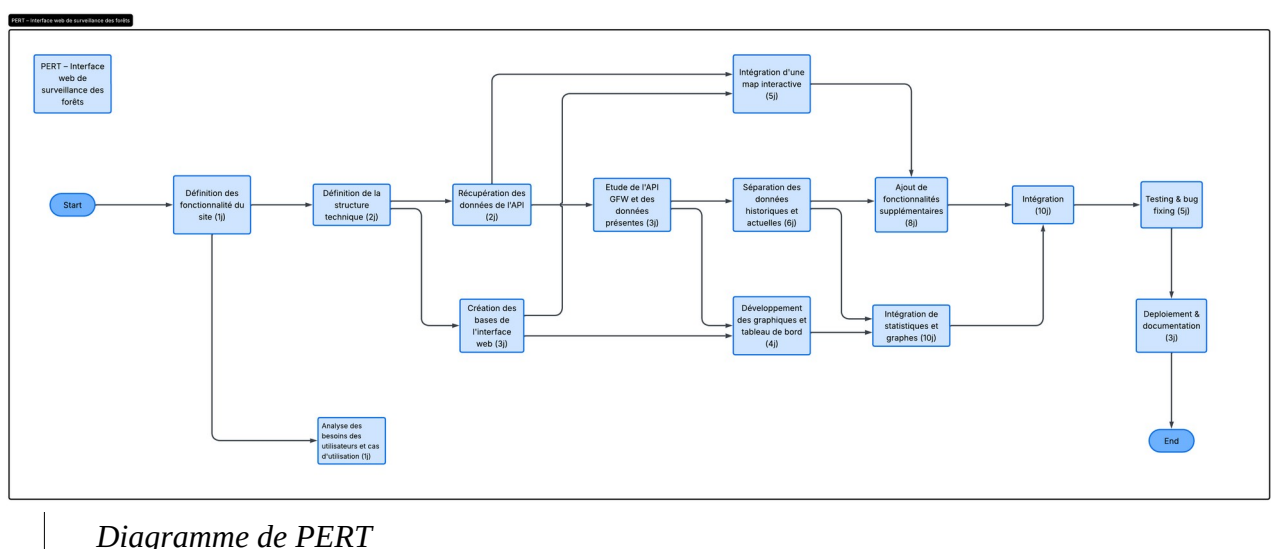


Diagramme de PERT:

L'architecture de notre gestion de projet s'articule autour de quatre grandes phases :

- **L'amorçage (Conception) :**

Le projet débute par un tronc commun séquentiel incluant la définition des fonctionnalités et de la structure technique (3 jours cumulés). Cette étape conceptuelle est un prérequis absolu avant toute répartition de l'écriture du code.

- **La parallélisation (Développement) :**

C'est la zone où l'équipe se divise de manière asynchrone. La création des bases de l'interface web (3j) et la récupération des données de l'API (2j) s'exécutent en parallèle. Le schéma montre clairement que les étapes majeures suivantes (Intégration de la map interactive et Développement des graphiques) sont des nœuds de dépendance croisés : elles nécessitent à la fois que le socle de l'interface soit prêt ET que l'étude des données API soit complétée.

- **La convergence (Consolidation) :**

Les différentes branches de fonctionnalités (séparation des données historiques/actuelles, ajout de surcouches sur la map) convergent vers deux grosses phases d'intégration spécifiques (8 et 10 jours). C'est le point de rencontre logique où les données backend viennent alimenter les composants frontend.

- **La finalisation (Qualité) :**

L'ensemble des flux techniques se rejoint sur un nœud d'intégration globale (10j). Cette étape conditionne directement le démarrage de la phase de tests et de corrections de bugs (5j), pour aboutir enfin au déploiement et à la documentation de livraison (3j).

Projet L2 S4 [Deforest]
Groupe [Pingouins]

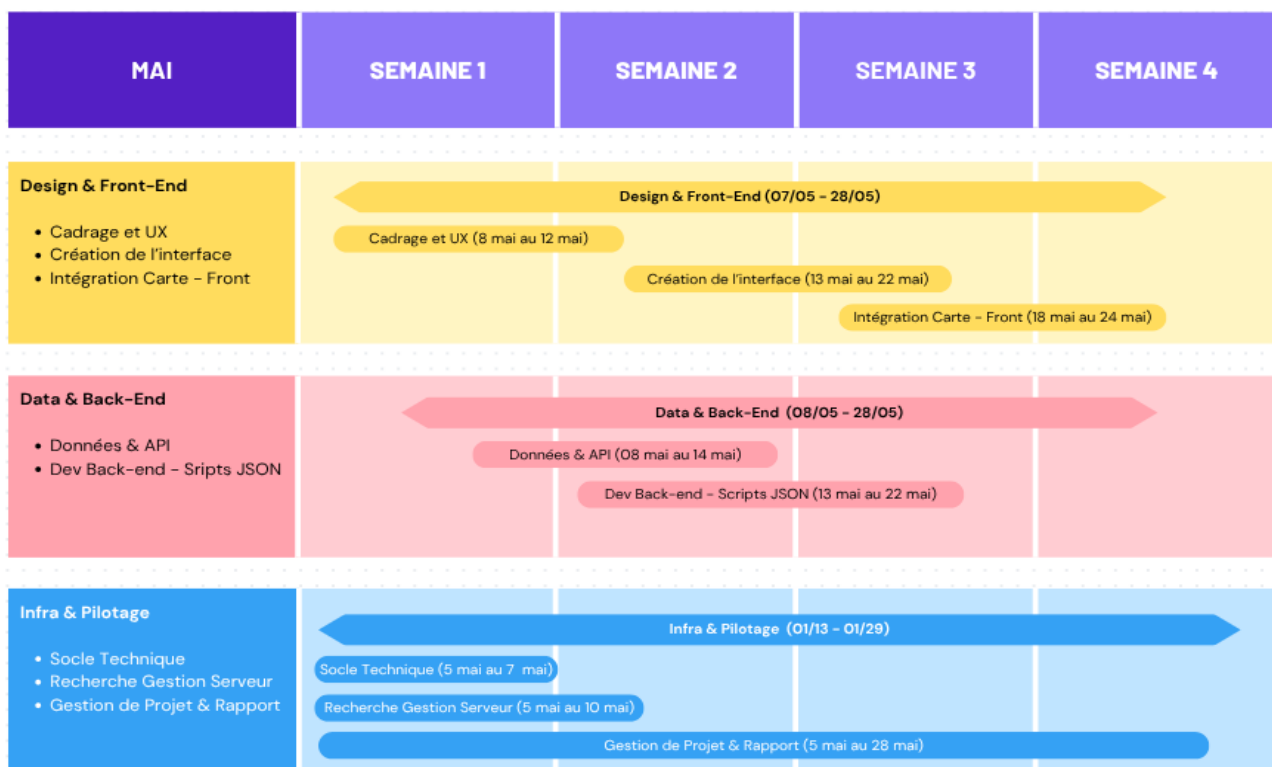


Diagramme de GANTT

Diagramme de GANT:

L'organisation s'est découpée en trois axes de travail simultanés :

- Infra & Pilotage (Le fil rouge) :** Le projet a démarré dès les premiers jours de mai par l'initialisation du socle technique (dépôts GitLab) et les recherches préalables sur la gestion du serveur. La coordination de l'équipe et la rédaction de ce rapport ont ensuite fait l'objet d'un suivi continu sur l'intégralité du mois.
- Data & Back-End :** En parallèle de la mise en place de l'infrastructure, l'exploration de l'API Global Forest Watch et de ses données a été amorcée (semaines 1 et 2). Cette phase de recherche a permis d'enchaîner avec le développement du backend (serveur Node.js et scripts de traitement JSON) sur les semaines 2 et 3.
- Design & Front-End :** Le travail sur l'interface a débuté par une phase de cadrage UX (semaine 1), suivie de la création de la structure web (semaines 2 et 3). L'intégration finale de la carte interactive est intervenue en semaine 3, au moment précis où le backend était structuré pour lui fournir les données environnementales.

- Outils mis en place sur le serveur

Cette section détaille l'architecture logicielle de notre application, les choix technologiques opérés pour répondre aux besoins du projet, ainsi que les stratégies de traitement de données mises en place pour contourner les limitations de notre source d'information principale.

Architecture et Technologies déployées

Pour ce projet, nous avons opté pour une architecture Full-Stack orientée micro-services, séparant clairement l'interface utilisateur de la logique serveur :

- Serveur (Back-end):

Développé en Node.js avec le framework Express.js. Il agit comme un pont sécurisé (proxy) entre notre interface web et les serveurs de Global Forest Watch.

- Base de données:

Nous avons fait le choix stratégique de ne pas héberger de base de données relationnelle locale (type MySQL). Face au volume colossal des données environnementales mondiales, l'API de GFW fait elle-même office de base de données que nous interrogeons en temps réel via des requêtes SQL encapsulées.

- Interface (Front-end):

Développée en HTML/CSS/JS natif, enrichie par Leaflet pour la cartographie spatiale et Chart.js pour la visualisation analytique.

Explication des points stratégiques du code

Afin de sécuriser et d'optimiser notre application, plusieurs mécanismes clés ont été implémentés côté serveur :

- Sécurisation par variables d'environnement : Le code ne contient aucune clé API en clair. L'authentification auprès de GFW (Bearer Token et API Key) est extraite d'un fichier .env protégé, garantissant que ces accès ne se retrouvent jamais exposés sur le dépôt GitLab public.
- Génération dynamique de requêtes SQL : Notre route principale (POST /api/alerts) est conçue pour recevoir les coordonnées géographiques (sud, ouest, nord, est) envoyées par la carte interactive. Le serveur Node.js injecte ensuite ces coordonnées dynamiquement dans une géométrie de type *Polygon* au sein d'une requête SQL (SELECT latitude, longitude...).

Analyse et Traitement des données

L'analyse des données se déroule en deux étapes pour transformer une donnée brute (des milliers de points JSON) en information pertinente pour l'utilisateur.

- Quoi ? Nous récupérons des tableaux de données contenant la position exacte des alertes de déforestation et leur niveau de confiance (confirmed ou suspected).
- Comment ? Le traitement est réalisé directement côté client (Front-end). Pour les widgets d'analyse, nous avons développé un algorithme de prédiction de tendance : le script évalue le volume d'alertes renvoyé (ex: si le total est supérieur à 300 alertes sur un échantillon donné, le système qualifie la tendance de "Hausse importante"). Pour l'affichage cartographique, nous utilisons un objet JavaScript Set (qui crée des clés uniques basées sur les latitudes/longitudes arrondies) pour filtrer les doublons stricts.
- Pourquoi ? Ce double traitement répond à un enjeu de performance et d'expérience utilisateur (UX). Filtrer les coordonnées en double avec un Set et utiliser le regroupement visuel (Clustering via Leaflet) permet d'éviter la surcharge de la mémoire vive du navigateur et garantit une navigation fluide, même lors de l'analyse de forêts denses. Les widgets d'analyse traduisent ensuite ces chiffres bruts en indicateurs écologiques compréhensibles (ex: Tendance stable ou alarmante).

Défi technique : Les limites de l'API et le traitement différé

Le principal obstacle technique de ce projet a résidé dans l'exploitation de l'API Global Forest Watch. Les données environnementales mondiales représentent un volume massif.

Le problème : Lors de sélections de zones géographiques trop vastes (ou de dézooms importants sur la carte), la requête SQL générée par notre serveur oblige l'API GFW à parcourir des millions de lignes d'historique. Ce traitement s'avère beaucoup trop long côté serveur distant. Conséquence : la requête subit un délai d'attente dépassé (Timeout), et l'API nous renvoie une erreur critique (Code 500) au lieu des données.

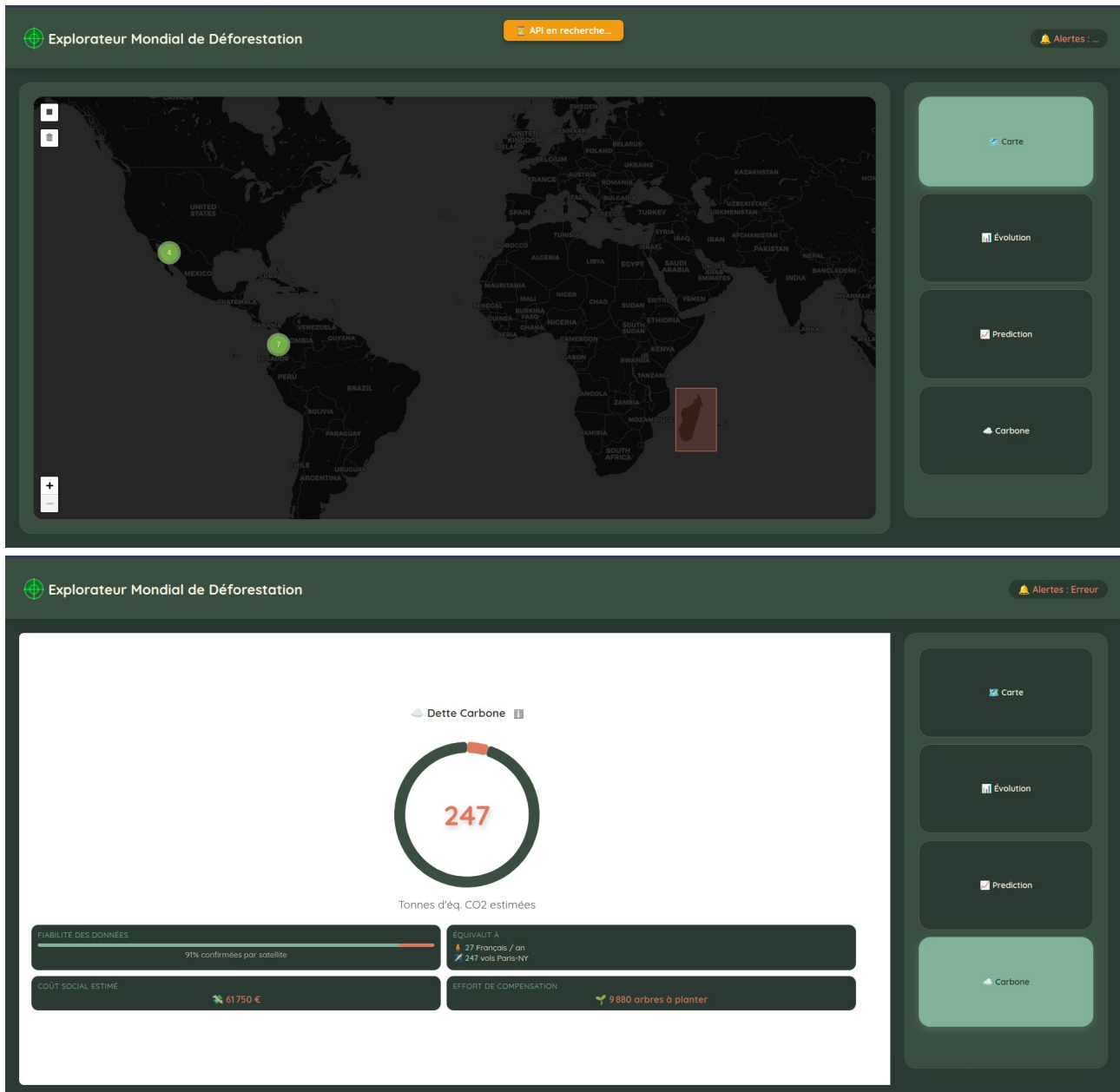
La solution implémentée : Pour pallier ce problème d'infrastructure indépendant de notre volonté, nous avons mis en place deux stratégies de contournement :

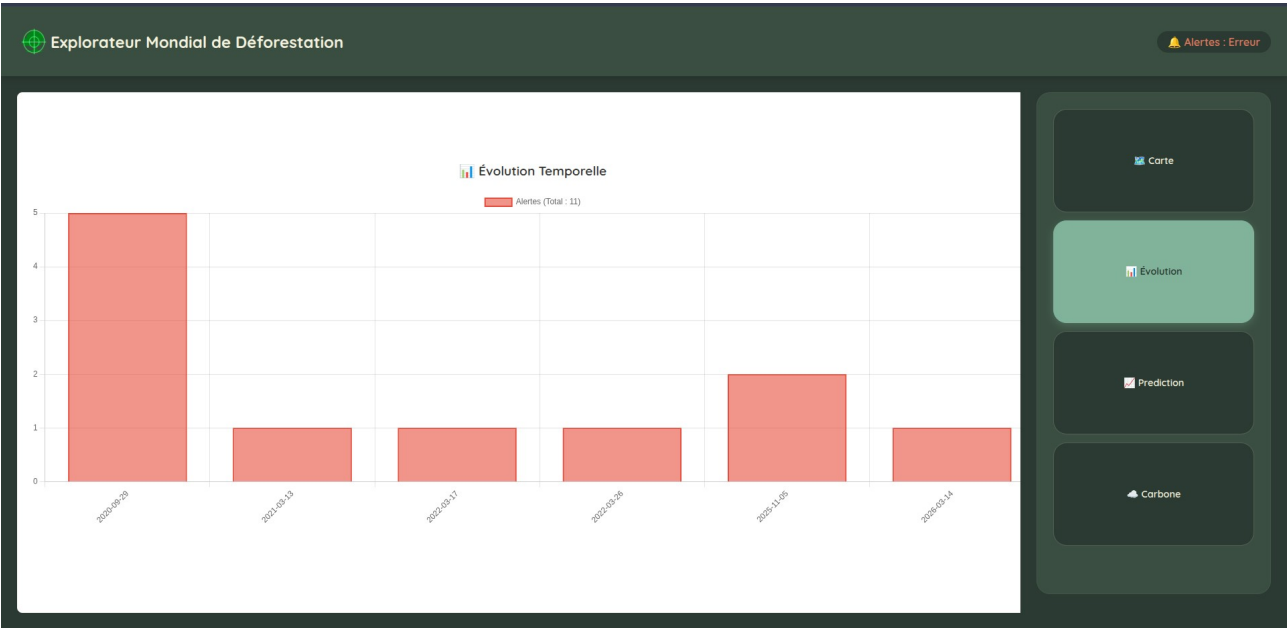
1. Limitation de la charge SQL : Nous avons volontairement ajouté une instruction LIMIT dans notre requête backend pour restreindre le nombre de résultats maximum renvoyés, évitant ainsi un crash systématique lors du chargement initial de la carte monde.
2. Gestion asynchrone des erreurs (UI) : Au lieu de laisser l'application "planter" silencieusement, nous avons entouré nos appels fetch de blocs try...catch couplés à une interface dynamique. Si l'API met trop de temps à répondre ou plante, l'erreur est interceptée et l'utilisateur est immédiatement averti par une bulle de statut visuelle rouge ("Erreur retournée par l'API") au-dessus de la carte.

3.

- Interface Utilisateur

Le résultat de cette architecture est un tableau de bord interactif, robuste face aux erreurs réseau.





V - Répartition des Tâches & Avancement

Voici la répartition globale des taches:

Description de l'état:

Fait
En cours
Pas commencé

Tableau de Répartition:

Les responsables sont trié par quantité de travail en fonction de chaque domaine de taches.

Domaine de Tâches	Description de la tâche	Responsable
Cadrage et UX	Création des maquettes de l'interface visuelle	1. Zambrano Junior 2. Ratsimbazafy Harinavalona

Création de l'interface	Création de l'interface web	1. Zambrano Junior 2. Ratsimbazafy Harinaivalona
Données & API	Exploration de l'API GFW (utilisation, création d'une clé, etc...)	1. Fernandez Mathys 2. Aballo Zaef 3. Soilihi Timéo
Socle Technique	Initialisation du projet (dépôts gitlab)	1. Fernandez Mathys
Dev - Back-end	Écriture des scripts pour récupérer les données de GFW	1. Fernandez Mathys 2. Aballo Zaef 3. Soilihi Timéo
Dev - Front (Carte)	Intégration de la librairie cartographique.	1. Zambrano Junior 2. Soilihi Timéo 3. Aballo Zaef
Rechercher Gestion Serveur	Recherche sur la gestion du serveur	1. Soilihi Timéo 2. Abbes Yiris
Gestion de Projet	Suivi du Gantt, organisation des réunions, rédaction de ce rapport.	1. Fernandez Mathys

VI - Rapports Personnels

Fernandez Mathys

Dans notre groupe de six, j'ai pris le rôle de chef de projet tout en participant activement au code (sur la partie Full-Stack). Ma mission principale était de gérer le cœur de l'application : la connexion avec l'API Global Forest Watch (GFW).

Mon travail s'est divisé en trois grandes parties.

Gestion de projet et versioning (GitLab):

Pour qu'on puisse avancer sans se marcher sur les pieds, j'ai organisé notre dépôt GitLab :

- **Planification** : J'ai mis en place les *Milestones* pour qu'on ait des étapes claires et qu'on respecte la date de rendu.
- **Répartition du travail** : J'ai créé et assigné les *Issues* pour que chacun sache exactement ce qu'il avait à faire.
- **Gestion du code (Merge)** : Je me suis occupé de fusionner les branches de tout le monde. J'ai notamment dû gérer les conflits de *merge* sur GitLab (comme lorsqu'il a fallu fusionner le système de clustering de la carte avec les bulles de statut), pour être sûr que la branche principale *dev* reste toujours propre et fonctionnelle.

Prise en main de l'API Global Forest Watch:

Comme la récupération des données était le point central du projet, j'ai passé une bonne partie du temps à comprendre comment l'API fonctionnait :

- **Compréhension technique** : J'ai épluché la documentation de GFW pour comprendre comment structurer nos requêtes (notamment le fait d'envoyer du SQL via des appels POST).
- **Code Backend** : J'ai implémenté les appels avec *fetch* dans notre fichier serveur, pour que notre backend interroge l'API avec les bonnes coordonnées quand l'utilisateur dessine une zone.

Traitement des données et affichage (Frontend):

Une fois les données reçues sur le client, il fallait s'assurer qu'elles s'affichent correctement sans faire ralentir la page :

- **Tri des données** : J'ai géré l'extraction du JSON renvoyé par l'API. J'ai aussi aidé à mettre en place un filtre (avec un objet *Set*) pour éviter d'afficher plusieurs fois les mêmes coordonnées et alléger le rendu de la carte.
- **Interface (UI)** : J'ai codé un système de bulles de statut pour que l'utilisateur comprenne ce qui se passe. En utilisant des bloc *'try catch'*, le code gère proprement les erreurs (comme les plantages 500 ou les problèmes d'autorisation 403) et affiche des messages clairs (Recherche en cours, Succès, Aucun résultat, Erreur) au lieu de simplement renvoyer des erreurs invisibles dans la console.

Conclusion :

Ce projet m'a permis de toucher un peu à toutes les couches d'une application web. Ça m'a vraiment fait progresser en JavaScript (surtout sur l'asynchrone et les promesses) et sur la manipulation d'API; Surtout, le fait d'agir comme chef de projet m'a donné une vraie expérience de

coordination d'équipe avec Git, ce qui correspond tout à fait à ce que j'aimerais faire plus tard dans la gestion de projets informatiques ou le Big Data.

Ratsimbazafy Harinavalona

Soilihi Timéo

Dans le cadre de ce projet, j'ai travaillé sur plusieurs aspects, allant de la gestion du serveur jusqu'à l'amélioration de l'interface utilisateur, afin de rendre l'application à la fois fonctionnelle et plus agréable à utiliser.

Mise en place du serveur et déploiement

Dans un premier temps, je me suis occupé de toute la partie hébergement pour que notre projet soit accessible en ligne. J'ai configuré un serveur web Apache, ce qui m'a permis de déployer notre projet. Pour gérer l'arborescence et envoyer nos fichiers locaux vers le serveur, j'ai utilisé le client FTP FileZilla, qui est une application qui permet de transférer les fichiers facilement par glisser-déposer. Cela nous a permis de mettre notre projet en ligne simplement.

Automatisation de l'environnement de travail

Afin d'optimiser notre flux de travail collaboratif, nous utilisons l'extension Visual Studio Code Remote - SSH. Au lieu de développer en local puis de transférer les fichiers, cet outil nous permet de nous connecter et de coder directement sur le serveur distant. Chaque sauvegarde met instantanément à jour le code sur le serveur, éliminant ainsi le besoin de transferts manuels (via FTP/SFTP) et réduisant le risque de perte de données.

Amélioration de la carte et correction des bugs d'affichage

La plus grosse partie de mon travail sur le code a été de rendre la carte interactive beaucoup plus propre. Au début, on rencontrait un problème avec Leaflet : quand plusieurs alertes avaient exactement les mêmes coordonnées (notre API nous donne parfois plusieurs fois le même point à des dates différentes), le plugin créait un effet "toile d'araignée" (spiderfy) très géométrique et pas

du tout naturel. Quand on zoomait au maximum, il affichait les points à égale distance. Pour corriger cela, j'ai désactivé cet effet artificiel et j'ai développé un algorithme de filtrage (en utilisant un objet `Set` en JavaScript). Désormais, si plusieurs points tombent exactement au même endroit, le code les fusionne en mémoire pour n'en afficher qu'un seul. Cela évite de surcharger l'écran et la carte reste lisible, même quand on dézoome.

Ergonomie et interface utilisateur

Enfin, j'ai voulu simplifier la vie de l'utilisateur lors de ses recherches sur la carte. Au départ, on devait zoomer manuellement sur une zone précise pour afficher les points liés aux lieux de déforestation, ce qui n'était pas très pratique. J'ai donc créé une fonctionnalité de sélection "drag and draw" (similaire à une capture d'écran) qui va afficher tous les points présents dans cette zone précise. J'ai également développé un bouton "Poubelle", intégré directement dans la barre d'outils de la carte. Un seul clic suffit pour tout réinitialiser : il efface le rectangle de sélection, retire tous les points rouges, remet le compteur d'alertes à zéro et vide la mémoire du programme pour les recherches suivantes. Cela évite d'avoir à recharger la page.

Abbes Yiris

Zambrano Junior

Premier partie l'index

Pour la première étape, je me suis occupé du fichier `index.html`. En fait, c'est la structure de base du tableau de bord, donc j'ai essayé de faire ça assez lisiblement. Au début, j'avais tendance à utiliser des balises `<div>` un peu partout, mais on s'y perd vite quand on cherche un bug. Du coup, j'ai organisé le code avec des balises plus spécifiques : j'ai placé un `<header>` pour la partie haute, un grand `<main>` pour contenir l'espace de la carte, et un `<aside>` pour regrouper les trois widgets sur le côté droit.

Ensuite, il a fallu gérer le style CSS. Au lieu de faire un seul fichier énorme qui contrôle tout le site, j'ai préféré créer plusieurs petits fichiers séparés (un pour la carte, un pour l'évolution, le global, etc.). Comme on travaille en groupe, ça limite pas mal les problèmes au moment de mettre nos codes en commun sur GitLab. Ça évite les conflits où deux personnes modifient le même gros fichier en même temps.

Il y avait aussi l'intégration des bibliothèques externes, c'est-à-dire Leaflet pour afficher la carte et Chart.js pour le graphique. Le souci avec ces gros scripts, c'est que ça peut ralentir l'affichage de la page. Donc, j'ai simplement ajouté l'attribut `defer` quand je les ai importés. Ça permet au navigateur d'afficher le visuel de la page en premier, pour ne pas laisser l'utilisateur bloqué devant un écran blanc pendant que les scripts finissent de se charger en arrière-plan.

Pour finir, même si on n'avait pas encore les données du serveur pour remplir les widgets, j'ai préparé les espaces dans le HTML. J'ai ajouté des id précis sur les zones de texte vides, par exemple `id="compteur-carbone"`. L'idée c'était de s'avancer un peu : quand il faudra injecter les vrais chiffres avec JavaScript plus tard, ce sera beaucoup plus direct pour cibler les bons éléments.

Deuxième partie `global.css`

Après le HTML, il fallait s'occuper de l'apparence. J'ai commencé par le fichier `global.css`. Une des premières étapes a été d'importer la police d'écriture, *Quicksand*. J'aurais pu simplement indiquer le nom de la police dans le code, mais si l'utilisateur ne l'a pas installée sur son ordinateur, le rendu visuel n'aurait pas fonctionné comme prévu. Pour éviter cela, j'ai utilisé la règle `@font-face` pour lier directement les fichiers `.ttf` que nous avons téléchargés dans notre dossier. Cela permet de conserver l'esthétique du site sur n'importe quel écran.

Ensuite, pour gérer les couleurs, cela devenait vite répétitif de copier les codes comme `#2B3A32` un peu partout, avec toujours le risque de se tromper d'un caractère. J'ai donc préféré utiliser des variables CSS dans le `:root`. J'y ai défini nos couleurs principales (le vert forêt, le terracotta pour les alertes) ainsi que les arrondis des éléments. L'avantage de cette méthode, c'est que si l'on décide finalement que le vert est trop sombre, il suffit de le modifier à cet unique endroit pour que tout le site se mette à jour.

Pour la mise en page, aligner correctement la carte avec les statistiques a demandé un peu de réflexion. J'ai finalement opté pour un `display: grid` sur le conteneur principal. Concrètement, j'ai fixé la largeur de la barre de droite (les widgets) à `350px` pour garantir sa lisibilité, et j'ai configuré l'espace de la carte pour qu'il prenne le reste de l'écran en utilisant l'unité `1fr`. Cela donne le maximum de place à la carte sans cacher les autres informations. À l'intérieur des widgets eux-mêmes, j'ai plutôt utilisé `flexbox` pour empiler les titres et les données correctement sans avoir de problèmes de décalage avec les marges.

Troisième partie `carte.js` et `carte.css`

Pour cette troisième étape, je me suis concentré sur l'affichage de la carte interactive en utilisant la bibliothèque Leaflet. Dans le fichier `carte.js`, j'ai initialisé la carte en la centrant sur des coordonnées globales de départ.

Il a fallu ensuite choisir le visuel de la carte. J'ai configuré les tuiles pour utiliser le fournisseur CartoDB avec leur thème "Dark Matter". Ce fond sombre s'intègre beaucoup mieux avec le design du tableau de bord, et cela permettra surtout de bien faire ressortir les futurs points de données. En manipulant l'interface, je me suis rendu compte que la carte se comportait un peu comme un globe : les continents se répétaient indéfiniment si on allait trop sur les côtés. Pour corriger ça, j'ai tout simplement ajouté l'option `noWrap: true` pour couper cette répétition.

Ensuite, pour éviter que l'utilisateur ne se perde dans l'espace vide en glissant la souris trop vers le nord ou le sud, j'ai défini des limites de coordonnées strictes avec `maxBounds`. J'ai couplé ça avec un `maxBoundsViscosity` à 1.0 pour vraiment bloquer fermement la caméra sur les bords et empêcher tout dépassement. Concernant le zoom, je l'ai configuré pour qu'il commence et se bloque à un minimum de 3. Au début, j'avais laissé un zoom plus éloigné, mais ça créait des bandes vides sur les côtés : la carte avait l'air un peu coupée et ne s'accommodait pas bien avec l'espace que je lui avais réservé dans l'interface. J'ai aussi mis un maximum à 18 pour éviter que les tuiles ne deviennent floues si on s'approche trop. L'idée globale, avec toutes ces limites, c'était de garder un contrôle total sur l'exploration pour que l'affichage reste toujours propre et bien encadré.

En observant l'interface par défaut, j'ai aussi remarqué que certains éléments méritaient d'être réorganisés. Leaflet affiche automatiquement ses boutons de zoom en haut à gauche et un texte d'attribution en bas à droite. J'ai préféré désactiver le texte d'attribution sur l'écran pour garder un visuel très net, bien que j'aie conservé les crédits officiels (OpenStreetMap et CARTO) directement dans la configuration du code pour respecter les licences. Concernant les boutons de zoom, je les ai désactivés en haut pour les recréer en bas à gauche de l'écran. Vu que la barre des statistiques prend déjà toute la partie droite, placer ces boutons à gauche permet d'équilibrer l'espace.

Enfin, j'ai fait un dernier ajustement dans un fichier séparé, `carte.css`. Quand on atteignait la limite sud de la carte, sous l'Antarctique, il y avait parfois un espace vide qui s'affichait avec un fond gris clair par défaut. Pour corriger cela, j'ai modifié la couleur de fond du conteneur de Leaflet pour mettre un noir très profond (`#0f0f0f`). Grâce à cela, l'océan sombre se fond parfaitement avec le fond du site.

Aballo Zaef

Présentation

Dans le cadre du projet transversal Deforest, j'ai principalement travaillé sur l'intégration des données API, l'affichage des points de déforestation sur la carte, la mise en place du clustering, la sécurisation des clés API ainsi que la documentation technique de l'API.

Mise en place des marqueurs de déforestation

J'ai développé la partie permettant d'ajouter des points de déforestation sur la carte Leaflet. J'ai créé une fonction JavaScript permettant d'ajouter dynamiquement des marqueurs, d'afficher des popups contenant des informations et de préparer l'intégration des données réelles provenant de l'API Global Forest Watch.

Mise en place du clustering des marqueurs

Afin d'améliorer la lisibilité de la carte lorsque plusieurs alertes apparaissent dans une même zone, j'ai intégré le système de clustering avec le plugin Leaflet MarkerCluster. Cette fonctionnalité

permet de regrouper automatiquement les points proches, d'éviter une surcharge visuelle et d'améliorer les performances de la carte.

Sécurisation des clés API

J'ai participé à la mise en place d'une architecture backend sécurisée utilisant Node.js, Express.js, dotenv et CORS. Le frontend communique désormais avec un backend Express qui interroge ensuite l'API Global Forest Watch. Les clés API sont stockées dans un fichier .env afin de ne pas être visibles dans le navigateur.

Mise en place du backend Express.js

J'ai participé à la création du backend Express.js utilisé comme intermédiaire entre le frontend et l'API GFW. Les fonctionnalités réalisées comprennent la création du serveur backend, la gestion des routes API, la récupération des données JSON ainsi que la gestion des erreurs.

Documentation API

J'ai rédigé une documentation technique de l'API afin d'expliquer l'architecture backend, l'installation de Node.js, l'utilisation de npm, le rôle du fichier .env et le fonctionnement des endpoints API.

Utilisation de Git et GitLab

Tout mon travail a été réalisé avec Git et GitLab à travers des branches dédiées, plusieurs commits structurés et des Merge Requests afin de conserver un historique clair des modifications apportées au projet.

Conclusion

Les tâches que j'ai réalisées ont permis d'améliorer l'interactivité de la carte, de préparer l'intégration des données réelles de déforestation, de sécuriser les accès API et de mettre en place une architecture backend plus professionnelle.

VII - Pourquoi ce cas d'utilisation ?

VIII – Referiez-vous le projet de la même manière ?
